

Week 07: Functor and applicative parsers

Functional Programming in Lean

Learning goals

- ▶ Represent a parser as a function
- ▶ Map a parsed value without consuming more input
- ▶ Sequence parsers that consume input in order

Central idea

A parser returns a value and the unconsumed input, or fails.

Lean example

```
abbrev Parser ( $\alpha$  : Type) :=  
  List Char  $\rightarrow$  Option ( $\alpha \times$  List Char)  
  
def character (wanted : Char) : Parser Char  
| c :: rest =>  
  if c == wanted then some (c, rest)  
  else none  
| [] => none
```

What to notice

- ▶ Mapping changes only the result
- ▶ Sequencing passes remaining input to the next parser
- ▶ The same operations motivate Functor and Applicative

Live coding

1. Complete character
2. Write mapParser
3. Combine two character parsers

Pause and predict

What remains after parsing ab from abc?

Exercise connection

Open `exercises/Week07.md` after the lecture.

The complete example is `Course/Week07.lean`; the fill-in version is `Templates/Week07.lean`.